МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РЕСПУБЛИКИ КАЗАХСТАН
Управление образования города Алматы
ТОО «Колледж Хекслет»

«УТВЕРЖДЕНЫ»
Решением УМС
Председатель _____
«___» _____ 20___ г., протокол № ___

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ ПО ВЫПОЛНЕНИЮ КУРСОВОЙ РАБОТЫ ПО СПЕЦИАЛЬНОСТИ: 06130100 ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ (ПО ВИДАМ) ПО ПМ4 «РАЗРАБОТКА БРАУЗЕРНЫХ ПРИЛОЖЕНИЙ»

Рассмотрены
на заседании ПЦК «Программное обеспечение»
Протокол № ___ от «___» _____ 20___ г.

Алматы
2026

1. Общие положения

1.1. Курсовая работа является самостоятельной проектной работой обучающегося, выполняемой в рамках ПМ4 «Разработка браузерных приложений» под руководством преподавателя.

1.2. Цель выполнения курсовой работы — систематизация и практическое применение знаний по разработке серверных приложений Node.js, Express.js, созданию REST API, моделированию реляционной базы данных PostgreSQL с использованием Prisma ORM, аутентификации и авторизации пользователей.

1.3. В результате выполнения курсовой работы обучающийся должен разработать функционирующее серверное приложение с REST API, оформить пояснительную записку, подготовить документацию README.md и опубликовать исходный код в репозитории GitHub.

1.4. Курсовая работа направлена на развитие навыков анализа предметной области, проектирования данных, программной реализации, проверки безопасности, тестирования и документирования программного обеспечения.

1.5. Обязательными технологиями являются Node.js (JavaScript с ES Modules), Express.js, Prisma ORM, PostgreSQL и Git/GitHub. Применение других технологий допускается как дополнение, но не вместо обязательных.

1.6. Студент отвечает за самостоятельность выполнения, достоверность результатов тестирования, соблюдение требований к информационной безопасности и воспроизводимость проекта по инструкции.

2. Требования к тематике курсовой работы

2.1. Тематика курсовых работ должна соответствовать специальности и профессиональному модулю, быть прикладной, актуальной и предусматривать разработку REST API для конкретной предметной области.

2.2. Обучающийся выбирает тему из перечня, утверждённого в образовательной организации, либо предлагает собственную тему, согласовывая её с преподавателем в установленном порядке.

2.3. Рекомендуемая формулировка темы: «Разработка REST API для [наименование или назначение приложения]». Например: «Разработка REST API для системы управления библиотекой».

2.4. Примерные направления: управление задачами, каталог товаров, учёт оборудования, система мероприятий, библиотечная система, запись на занятия, личные финансы, каталог фильмов и сериалов.

2.5. Рекомендуемый объём модели данных — 3–5 сущностей, включая User; иное количество допускается с обоснованием и согласованием. Обязательна связь не менее двух сущностей посредством внешнего ключа.

2.6. В рамках любой темы должны быть реализованы регистрация, вход с JWT, роли user и admin, разграничение доступа к ресурсам, полный CRUD основной сущности и запрос связанных данных.

2.7. До начала реализации студент согласовывает постановку задачи, предварительную ER-диаграмму, перечень HTTP-маршрутов и матрицу прав доступа.

3. Содержание и структура курсовой работы

3.1. Курсовая работа должна соответствовать требованиям к содержанию, практической реализации и оформлению, установленным образовательной организацией.

3.2. Структурными элементами пояснительной записки являются:

– **титульный лист** (1 стр.);

– **содержание** (1 стр.);

– **введение** (1);

– **Глава 1**. Теоретическая часть — анализ и проектирование REST API (10 стр..);

– **Глава 2**. Практическая часть — реализация и тестирование REST API (10 стр.);

– **заключение** (2 стр.);

– **список использованных источников**;

– **приложения** (при необходимости).

- Общий объём курсовой работы должен составлять не менее 25 страниц печатного текста без учёта приложений.

3.3. Титульный лист является первой страницей курсовой работы. На нём приводятся название образовательной организации, название и тема работы, модуль, специальность, квалификация, сведения о студенте и преподавателе, отметки допуска и защиты, город и год. Образец приведён в Приложении 1.

3.4. Содержание должно включать названия структурных элементов, разделов и подразделов с указанием страниц. Образец приведён в Приложении 2.

3.5. Во введении необходимо обосновать актуальность темы, сформулировать цель, задачи, объект и предмет разработки, обозначить применяемые технологии и ожидаемый результат.

3.6. **Глава 1.** Теоретическая часть (анализ и проектирование REST API). В данном разделе необходимо:

1. Провести анализ предметной области: назначение приложения, категории пользователей, основные сценарии использования, функциональные и нефункциональные требования.

2. Спроектировать базу данных: выделить сущности и их поля, объяснить первичные и внешние ключи, построить ER-диаграмму со связями и кратностями.

3. Спроектировать REST API: описать методы и URL, параметры, структуры JSON-запросов и ответов, успешные статусы и возможные ошибки.

4. Составить матрицу доступа guest / user / admin, описать правила проверки JWT и ограничения для операций с данными пользователя.

5. Обосновать выбор Node.js, Express.js, Prisma ORM, PostgreSQL и организации кода по слоям.

3.7. При проектировании необходимо отдельно отразить обязательные функции:

– полный CRUD основной сущности — создание, список, получение по ID, изменение и удаление;

– чтение другой сущности и хотя бы один запрос связанных данных посредством Prisma include или select;

– модель User (уникальный email, имя, роль, хеш пароля), регистрация и вход;

– роли user и admin и ограничения административных маршрутов на сервере;

– валидация входных данных и централизованная обработка ошибок HTTP.

3.8. Глава 2. Практическая часть (реализация и тестирование REST API). В данном разделе необходимо:

1. Описать структуру проекта, назначение каталогов routes, controllers, services, middleware, config и prisma; показать схему взаимодействия Route → Controller → Service → Prisma → PostgreSQL.

2. Привести значимые фрагменты Prisma Schema и порядок подготовки базы PostgreSQL; продемонстрировать применение внешних ключей и получение связанных данных.

3. Описать реализацию HTTP-маршрутов, обработчиков CRUD и форматов JSON; привести примеры корректных запросов и ответов.

4. Показать реализацию регистрации, безопасного хеширования паролей (например, Argon2id), login, выдачи JWT, middleware requireAuth и requireRole.

5. Объяснить валидацию данных, обработку ошибок (400, 401, 403, 404, 409, 500) и защиту конфиденциальной информации от попадания в ответы и репозиторий.

6. Выполнить тестирование REST API: позитивные и негативные сценарии CRUD, проверка связей, регистрации, входа и разграничения доступа.

7. Описать README.md, публикацию на GitHub, настройку .env.example и проверку запуска проекта на другом компьютере.

3.9. В README.md обязательно документируется каждый фактически реализованный endpoint: метод и URL, назначение, условия доступа и роли, параметры, заголовки, пример Request Body, пример Response Body, коды успешного и ошибочных ответов.

3.10. В README.md и пояснительной записке должна присутствовать одна и та же актуальная ER-диаграмма с обозначениями PK/FK и связей. Для защищённых маршрутов следует показать заголовок Authorization: Bearer <accessToken> с условным токеном.

3.11. При работе с пользовательскими ресурсами владелец определяется из подтверждённого req.user.id, а не из переданного клиентом userId. При публичной регистрации нельзя самостоятельно назначать роль admin. В ответе запрещено возвращать passwordHash.

3.12. В заключении кратко фиксируются достигнутые результаты, соответствие поставленным задачам, выявленные ограничения и направления дальнейшего развития проекта.

3.13. В список использованных источников включаются реально использованные публикации и официальная документация Node.js, Express, Prisma, PostgreSQL, JWT и инструментов тестирования. Библиографические требования определяются локальными правилами колледжа.

3.14. В приложения при необходимости включаются ER-диаграмма, матрица доступа, дополнительные скриншоты тестирования, примеры запросов и ответов, фрагменты исходного кода. Полные листинги рекомендуется хранить в репозитории.

3.15. Дополнительные функции (refresh token, восстановление пароля, Jest/Supertest, Docker, CI, деплой, мониторинг, frontend) допускаются по выбору и не заменяют обязательную функциональность.

4. Порядок защиты курсовой работы

4.1. В ходе защиты обучающийся должен продемонстрировать понимание предметной области, архитектуры приложения, принципов REST и организации безопасного доступа к ресурсам.

4.2. Процедура защиты включает доклад обучающегося, демонстрацию работоспособности проекта, проверку документации и ответы на вопросы комиссии.

4.3. Продолжительность доклада рекомендуется устанавливать в пределах 5–7 минут; презентация должна включать не более 15 слайдов.

4.4. В докладе последовательно раскрываются актуальность, цель и задачи; ER-диаграмма; структура проекта; пример CRUD и связанного запроса; регистрация и login; роль JWT и проверки прав; основные результаты тестирования.

4.5. При демонстрации необходимо показать работающие запросы GET, POST, PATCH, DELETE и коды ошибок, а также отдельно: запрос без JWT (401), попытку доступа user к ресурсу admin (403) и успешную операцию администратора.

4.6. Студент должен предъявить ссылку на GitHub-репозиторий и README.md с инструкциями запуска, ER-диаграммой, примерами HTTP-запросов и ответов. Работа должна запускаться согласно документации.

5. Оценивание курсовой работы

5.1. Максимальная оценка за курсовую работу составляет 100 баллов. Для оценивания используются следующие критерии, согласованные с техническим заданием ПМ4:

Критерий оценки	Максимальный балл	Полученный балл	Комментарий
1 Теоретическая часть (проектирование)	10		
1.1 Анализ предметной области, требований, сценариев	3		
1.2 ER-диаграмма, модель данных и связи	4		
1.3 Маршруты REST и матрица прав	3		
2 Практическая реализация	75		
2.1 Полный REST CRUD основной сущности	20		
2.2 Prisma ORM, PostgreSQL, связанные запросы	15		
2.3 Регистрация, хеширование паролей, login, JWT	15		
2.4 Авторизация, роли и проверка доступа	10		
2.5 Валидация, обработка ошибок, статусы HTTP	10		
2.6 Архитектура и качество кода	5		
3 Документирование и защита	15		
3.1 GitHub, README.md, воспроизводимость	10		
3.2 Демонстрация, тестовые сценарии и ответы	5		
ИТОГО	100		

5.2. Типичными существенными недостатками являются: отсутствие PostgreSQL, связи между сущностями или работающего CRUD; отсутствие регистрации, JWT и серверной проверки ролей; хранение открытых паролей; предоставление пользователю права назначить себе admin; отсутствие документации HTTP-методов и ER-диаграммы; раскрытие секретов в репозитории.

5.3. При оценивании учитываются фактическая работоспособность API, самостоятельность разработки, точность технических объяснений, корректность проектных решений и качество представленных материалов.

5.4. Оценка объявляется по итогам защиты в соответствии с установленной в колледже процедурой. При необходимости комиссия учитывает отзыв преподавателя и результаты проверки работы.

5.5. Буквенная система оценки учебных достижений обучающихся:

Буква	**Цифровой эквивалент**	**Проценты**	**Традиционная оценка**
A	4,0	95–100	Отлично
A−	3,67	90–94	Отлично
B+	3,33	85–89	Хорошо
B	3,0	80–84	Хорошо
B−	2,67	75–79	Хорошо
C+	2,33	70–74	Удовлетворительно
C	2,0	65–69	Удовлетворительно
C−	1,67	60–64	Удовлетворительно
D+	1,33	55–59	Удовлетворительно
D	1,0	50–54	Удовлетворительно
F	0	0–49	Неудовлетворительно

5.6. Оценка «ОТЛИЧНО» выставляется, если все обязательные функции работают, API документирован и воспроизводим, продемонстрированы корректная работа базы данных, JWT, разграничения прав и обработки ошибок, а обучающийся уверенно объясняет архитектурные решения.

5.7. Оценка «ХОРОШО» выставляется, если требования в целом выполнены, но имеются отдельные некритичные недочёты в реализации, документации, тестировании или объяснениях, не препятствующие основным сценариям.

5.8. Оценка «УДОВЛЕТВОРИТЕЛЬНО» выставляется, если работа частично отвечает требованиям, имеются существенные недостатки полноты реализации, проверки безопасности, проектирования или оформления, но представлен самостоятельный работающий результат.

5.9. Оценка «НЕУДОВЛЕТВОРИТЕЛЬНО» выставляется при отсутствии функционирующего проекта, существенном несоответствии теме, отсутствии ключевых обязательных компонентов или неспособности объяснить и защитить выполненную работу.

5.10. При неудовлетворительной оценке доработка и повторная защита организуются в порядке, установленном образовательной организацией.

Приложение 1
Образец оформления титульного листа

ТОО «Колледж Хекслет»

КУРСОВАЯ РАБОТА
на тему: «Разработка REST API для системы управления библиотекой»
по модулю: «Разработка браузерных приложений»
Специальность: 06130100 Программное обеспечение (по видам)
Квалификация: 4S06130105 Техник информационных систем

Выполнил(-а): студент(ка) ___ курса
группа _____
Ф.И.О. _____

Допущен(-а) к защите «___» _____ 20___ г.
Преподаватель: _____

Защитил(-а) с оценкой: _____
(буква) (цифр. экв.) (балл)

Алматы 2026

Приложение 2
Образец оформления содержания

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ТЕОРЕТИЧЕСКИЕ ЧАСТЬ: РАЗРАБОТКА АДАПТИВНОГО САЙТА ДЛЯ СТРОИТЕЛЬНЫХ КОМПАНИЙ 5

1.1 Анализ предметной области адаптивного сайта для строительной компании 5

1.2 Проектирование будущего интерфейса для адаптивного сайта для строительной компании 8

1.3 Выбор технологий для построения адаптивного сайта строительной компании 11

2 ПРАКТИЧЕСКАЯ ЧАСТЬ: СОЗДАНИЕ АДАПТИВНОГО САЙТА ДЛЯ СТРОИТЕЛЬНЫХ КОМПАНИЙ 15

2.1 Реализация HTML-разметки сайта 15

2.2 Реализация CSS-стилей сайта 20

2.3 Реализация JavaScript-функционала сайта 25

2.4 Тестирование сайта на соответствие поставленным требованиям и отработки функционала 30

ЗАКЛЮЧЕНИЕ 36

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 37

ПРИЛОЖЕНИ 38